

CoSpace

Informe de pruebas

Validación del backend y lógica de negocio

Proyecto: CoSpace

Tipo: humo + aceptación

Fecha: 17 de septiembre de 2026

Entorno: Windows / .NET 8 / xUnit

Documentación técnica de validación funcional del sistema

Índice general

1. Introducción	3
2. Objetivo	4
3. Entorno y herramientas	5
4. Metodología de pruebas unitarias	6
4.1. Diseño de la prueba	6
4.2. Repositorios falsos usados	6
4.3. Criterios de aceptación	6
4.4. Comandos relevantes	7
5. Prueba de humo	8
5.1. Definición	8
5.2. Comando ejecutado	8
5.3. Resultado esperado	8
5.4. Resultado obtenido	8
6. Pruebas de aceptación	9
6.1. Comando general	9
6.2. Casos ejecutados	10
6.3. Prueba filtrada en autenticación	10
6.4. Resultado esperado	10
6.5. Resultado obtenido	11
7. Resultados observados	12
7.1. Resumen ejecutivo	12
7.2. Interpretación	12
7.3. Cobertura por módulo	12
7.4. Evidencia de ejecución	12
7.5. Limitaciones conocidas	13
8. Conclusión	14

Capítulo 1

Introducción

La validación del sistema se realizó en dos niveles complementarios:

- **Prueba de humo:** verifica que el backend arranca correctamente y responde a peticiones HTTP sin errores.
- **Prueba de aceptación:** valida las reglas de negocio principales de autenticación, reservas y pagos.

En este proyecto, la API no se presenta como una interfaz visual en el navegador, sino como un servicio que recibe y devuelve respuestas HTTP. Por esta razón, la verificación se hizo principalmente desde la terminal, donde se ejecutaron los comandos de arranque, consulta y prueba automática.

Capítulo 2

Objetivo

El objetivo principal fue comprobar que:

- La API puede levantarse sin errores.
- La ruta principal responde correctamente.
- La lógica de negocio de registro, autenticación, reservas y pagos funciona como se espera.
- El sistema cumple con las reglas críticas definidas para la capa de aplicación.

Capítulo 3

Entorno y herramientas

- **Sistema operativo:** Windows
- **Framework:** .NET 8
- **Herramienta de pruebas:** xUnit
- **Entorno de ejecución:** PowerShell
- **Proyecto de pruebas:** `tests/CoSpace.Tests/CoSpace.Tests.csproj`

Capítulo 4

Metodología de pruebas unitarias

La validación de la lógica de negocio se realizó con pruebas unitarias aisladas, sin depender de servicios externos ni datos reales. El objetivo principal fue verificar que cada servicio cumpliera sus reglas sin requerir Firestore, Render ni Vercel.

4.1. Diseño de la prueba

Cada caso de prueba se construyó sobre servicios reales de aplicación, pero con repositorios falsos en memoria. Esto permitió probar la lógica sin alterar datos reales ni depender del entorno externo.

- **AuthService:** valida registro, login, normalización de campos y reglas de validación.
- **BookingService:** valida reservas, IVA, traslapes y rangos temporales inválidos.
- **PaymentService:** valida pagos, autorizaciones, facturas y prevención de duplicados.

4.2. Repositorios falsos usados

Se emplearon implementaciones simuladas de persistencia para aislar la lógica de negocio:

- `FakeUserRepository`
- `FakeBookingRepository`
- `FakePaymentRepository`

Esto hace que las pruebas sean deterministas, rápidas y reproducibles, además de evitar efectos secundarios sobre la base de datos real.

4.3. Criterios de aceptación

Una prueba se considera exitosa cuando:

- el servicio recibe los datos esperados,
- la salida cumple con la regla de negocio,

- la operación genera el efecto esperado,
- y se rechazan los casos inválidos con la excepción o error indicado.

4.4. Comandos relevantes

```
# Arranque de la API
dotnet run --project .\src\CoSpace.Api\CoSpace.Api.csproj --urls
http://localhost:5050

# Verificacion de respuesta HTTP
curl http://localhost:5050/api/sedes

# Ejecucion de pruebas de autenticacion
dotnet test .\tests\CoSpace.Tests\CoSpace.Tests.csproj --
configuration Release --logger "console;verbosity=normal" --
filter "FullyQualifiedNames~Auth"
```


Capítulo 5

Prueba de humo

5.1. Definición

La prueba de humo busca confirmar que el sistema inicia correctamente y responde a peticiones básicas sin errores de configuración, compilación o arranque. Es una validación rápida y esencial antes de ejecutar pruebas funcionales más profundas.

5.2. Comando ejecutado

```
curl http://localhost:5050/api/sedes
```

5.3. Resultado esperado

Se espera una respuesta HTTP exitosa con un cuerpo JSON que contenga la información de las sedes disponibles.

5.4. Resultado obtenido

La ejecución tuvo código de salida 0, indicando que la API respondió correctamente y que el backend quedó operativo en el puerto 5050.

Capítulo 6

Pruebas de aceptación

La prueba de aceptación valida que la lógica principal del sistema cumple con las reglas del negocio. En este caso, se centró en tres áreas clave:

- Autenticación
- Reservas
- Pagos

6.1. Comando general

```
dotnet test .\tests\CoSpace.Tests\CoSpace.Tests.csproj --  
configuration Release --logger "console;verbosity=normal"
```

6.2. Casos ejecutados

N°	Caso de prueba	Validación
1	RegisterAsync_creates_normalized_member_with_hashed_password	Normaliza nombre, correo y teléfono; crea el usuario y no almacena la contraseña en texto plano.
2	RegisterAsync_rejects_duplicate_email	Rechaza correos repetidos sin importar mayúsculas o minúsculas.
3	LoginAsync_accepts_correct_password_and_rejects_wrong_password	Acepta el login correcto y rechaza una contraseña incorrecta.
4	RegisterAsync_rejects_short_password	Exige una contraseña mínima de 8 caracteres.
5	ConfirmAsync_creates_booking_and_pending_payment_with_iva	Genera la reserva y el pago pendiente; calcula correctamente el IVA sobre el subtotal.
6	ConfirmAsync_rejects_overlapping_active_booking	Rechaza horarios que se cruzan con una reserva activa.
7	ConfirmAsync_allows_overlap_with_cancelled_booking	Permite la reserva si la reserva previa está cancelada.
8	ConfirmAsync_rejects_invalid_time_range	Rechaza un rango horario inválido cuando la hora final no es posterior a la inicial.
9	PayAsync_marks_pending_payment_as_paid_and_assigns_invoice	Actualiza un pago pendiente a pagado y le asigna la factura.
10	PayAsync_rejects_payment_by_different_user	Impide que un usuario distinto pague una reserva ajena.
11	PayAsync_returns_existing_paid_payment_without_new_invoice	Evita duplicidad de pagos y facturas repetidas.

Cuadro 6.1: Casos de prueba principales ejecutados durante la validación de aceptación.

6.3. Prueba filtrada en autenticación

Se ejecutó además una validación específica para la parte de autenticación:

```
dotnet test .\tests\CoSpace.Tests\CoSpace.Tests.csproj --
  configuration Release --logger "console;verbosity=normal" --
  filter "FullyQualifiedName~Auth"
```

6.4. Resultado esperado

Se esperaba que todas las pruebas del conjunto de autenticación se ejecutaran correctamente y regresaran una salida exitosa sin fallos.

6.5. Resultado obtenido

La ejecución regresó con código de salida 0, lo que confirma que el conjunto de autenticación quedó validado correctamente.

Capítulo 7

Resultados observados

7.1. Resumen ejecutivo

- La prueba de humo fue exitosa: la API respondió correctamente desde `http://localhost:5050/api/sed`
- La validación de autenticación tuvo resultado exitoso con salida 0.
- La suite de aceptación ejecutó 11 casos funcionales y no presentó fallos.

7.2. Interpretación

La prueba de humo demostró que la API no solo compiló, sino que además respondió a una petición HTTP real del sistema. La prueba de aceptación, por su parte, validó reglas de negocio esenciales que impactan directamente la experiencia del usuario final: creación de cuentas, acceso, reservas y pagos.

7.3. Cobertura por módulo

Módulo	Reglas validadas	Resultado
Autenticación	registro, correo duplicado, login correcto y fallido, contraseña corta	Exitosa
Reservas	cálculo de IVA, validación de horario, traslape activo, reserva cancelada	Exitosa
Pagos	pago pendiente, pago duplicado, pago por usuario incorrecto, factura asociada	Exitosa

Cuadro 7.1: Cobertura funcional por módulo de la lógica de negocio.

7.4. Evidencia de ejecución

- Comando ejecutado: `dotnet test .\tests\CoSpace.Tests\CoSpace.Tests.csproj -configuration Release -logger console;verbosity=normal`
- Resultado: salida exitosa con código 0.

- Validación de humo: `curl http://localhost:5050/api/sedes`
- Resultado: respuesta HTTP 200 con datos de sedes.

7.5. Limitaciones conocidas

Las pruebas realizadas corresponden a validación unitaria y funcional de la lógica de negocio. No sustituyen una ejecución completa en un entorno con Firebase real ni pruebas end-to-end con navegador. En particular, no se validaron:

- persistencia real en Firestore,
- conexión real a Render o Vercel,
- pasarela de pagos real,
- pruebas de carga o concurrencia simultánea,
- interacción visual completa con navegador.

Capítulo 8

Conclusión

La ejecución realizada confirma que el sistema cuenta con una validación funcional básica sólida para la etapa actual del proyecto. El backend respondió exitosamente en la prueba de humo y la lógica de autenticación, reservas y pagos se comportó de acuerdo con las reglas esperadas. La suite ejecutada cubre los principales escenarios de negocio del sistema y mostró resultados satisfactorios.

Capítulo 9

Anexo: comandos principales

```
# Prueba de humo
curl http://localhost:5050/api/sedes

# Prueba de aceptacion
dotnet test .\tests\CoSpace.Tests\CoSpace.Tests.csproj --
    configuration Release --logger "console;verbosity=normal"

# Prueba de autenticacion filtrada
dotnet test .\tests\CoSpace.Tests\CoSpace.Tests.csproj --
    configuration Release --logger "console;verbosity=normal" --
    filter "FullyQualifiedNames~Auth"
```